

EXP.NO: 8

Banker's Algorithm

DATE:13/03/2026

AIM:

To implement the Banker's Algorithm for deadlock avoidance, which determines whether the system is in a safe state before granting resource requests to processes.

ALGORITHM:

1. Start the experiment.
2. Input: number of processes (P), resource types (R), Allocation matrix, Maximum matrix, and Available vector.
3. Compute the Need matrix: $Need[i][j] = Maximum[i][j] - Allocation[i][j]$.
4. Initialize $Work = Available$ and $Finish[i] = false$ for all processes i .
5. Find process i such that $Finish[i] == false$ AND $Need[i] \leq Work$.
6. If found: set $Work = Work + Allocation[i]$, $Finish[i] = true$, append i to safe sequence, and repeat step 5.
7. If no such process exists and some $Finish[i] == false$, the system is in an unsafe state.
8. If all $Finish[i] == true$, print the safe sequence — the system is deadlock-free.
9. Stop the experiment.

CODE:

```
#include <stdio.h>

#define P 5    /* number of processes */
#define R 3    /* number of resource types */

int main() {
    int alloc[P][R] = { {0,1,0}, {2,0,0}, {3,0,2}, {2,1,1}, {0,0,2} };
    int maxm[P][R] = { {7,5,3}, {3,2,2}, {9,0,2}, {2,2,2}, {4,3,3} };
    int avail[R]    = { 3, 3, 2 };

    int need[P][R], finish[P] = {0}, safeSeq[P];
    int work[R];

    /* Compute Need matrix */
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = maxm[i][j] - alloc[i][j];

    for (int i = 0; i < R; i++) work[i] = avail[i];

    int count = 0;
    while (count < P) {
        int found = 0;
        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
```

```

        int ok = 1;
        for (int j = 0; j < R; j++)
            if (need[i][j] > work[j]) { ok = 0; break; }
        if (ok) {
            for (int j = 0; j < R; j++) work[j] += alloc[i][j];
            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }
}

if (!found) { printf("System is NOT in safe state!\n"); return 0; }

printf("System is in SAFE state.\n");
printf("Safe Sequence: ");
for (int i = 0; i < P; i++)
    printf("P%d%s", safeSeq[i], i < P-1 ? " -> " : "\n");

printf("\nNeed Matrix:\nP\tR0\tR1\tR2\n");
for (int i = 0; i < P; i++)
    printf("P%d\t%d\t%d\t%d\n", i, need[i][0], need[i][1], need[i][2]);

return 0;
}

```

OUTPUT:

```

[hp@localhost ~]$ gcc bankers.c -o bankers
[hp@localhost ~]$ ./bankers

```

```

System is in SAFE state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2

```

```

Need Matrix:
P      R0 R1 R2
P0     7  4  3
P1     1  2  2
P2     6  0  0
P3     0  1  1
P4     4  3  1

```

RESULT:

Thus, the Banker's Algorithm for deadlock avoidance was successfully implemented. The system determined the safe sequence P1 -> P3 -> P4 -> P0 -> P2 and confirmed the system is in a safe state with no possibility of deadlock.